

# DAY 2 (afternoon): Login with Google (OAuth 2.0), BACKEND

E-Learning Platform (Spring Boot 3.2.5, Java 17). **3 new files, 1 Maven dependency, 1 properties line.** `/api/login`, `/api/register`, `JwtUtil`, `AuthController` and the `User` table are not touched, so the SRS and hidden tests are not affected.

Do this **after the Day 2 instructor module** (about 3 hours). Person A: Google console plus this guide. Person B: the frontend guide.

## 0. What you get and the rules I chose

A "Continue with Google" button on the Login page. **Registration page:** "Sign up with Google" creates the account and stores it in the `users` table (e-mail, Google name, role CUSTOMER). **Login page:** "Sign in with Google" works **only for an e-mail that is already registered**; a person who never registered gets **404 "No account found for this Google e-mail. Please register first."** and nothing is stored. Signing up with an e-mail that already exists gives **409 "already registered, log in instead"**. Your own JWT and refresh cookie are issued exactly as for `/api/login`, so nothing after login changes.

| Rule                                                                                                                                                                                    | Why                                                                                                                      |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <b>Sign up with Google</b> (body <code>{credential, mode: "register"}</code> ) creates a <b>CUSTOMER</b> row; <b>Sign in with Google</b> (mode: "login", the default) never creates one | nobody can become ADMIN through Google                                                                                   |
| an existing <b>ADMIN</b> cannot use Google                                                                                                                                              | admins keep passwords (403). Registered customers and instructors may use it                                             |
| the same e-mail opens the <b>same account</b>                                                                                                                                           | Google already verified that e-mail                                                                                      |
| a new Google user gets the placeholder mobile <code>0000000000</code>                                                                                                                   | they can enter the real number on their profile page                                                                     |
| a person who <b>registered with the form and has not verified the phone yet</b> still has to verify                                                                                     | the OTP rule (Day 1) cannot be skipped through Google: the server answers 403 and the page sends them to the verify page |
| a brand-new Google user has no OTP row                                                                                                                                                  | Google verified their e-mail, so they count as verified                                                                  |
| empty <code>google.client-id</code>                                                                                                                                                     | button hidden, endpoint answers 400, the project works without any Google setup                                          |

**Honest status.** The **frontend** passed 14 checks (plus 3 for the OTP rule) in a real browser against a pretend backend and a stand-in Google button. The **Java backend was never compiled or run** here (Maven blocked); I only checked syntax. The **real Google pop-up was not run** because it needs your own Google project. Your first start downloads one new library; if you see a red error, send it to me.

## 1. OAuth explained from zero (tied to your project)

**Today (your JWT login).** You type email and password. `AuthController.login` checks them in the database and `JwtUtil.generateToken` creates a JWT signed with your own secret (`jwt.secret`).

Angular stores it and `AuthInterceptor` sends it as `Authorization: Bearer ...` on every request. `JwtRequestFilter` checks it.

**OAuth 2.0** is a set of rules that lets one company (Google) prove who a person is to another app (yours), without your app ever seeing the person's Google password. **OpenID Connect (OIDC)** is the identity layer on top of OAuth 2.0. Google uses it, and it gives your app an **ID token**.

**The four players** | Name | In your project | |---|---| | User (resource owner) | the person at the login page | | Client | your Angular app | | Authorization server | Google (accounts.google.com) | | Resource server | your Spring Boot API |

**The ID token** is also a JWT, but it is signed by Google with Google's **private** key (RS256). Your own JWT is signed with a shared secret (HS256). Because Google publishes its **public** keys at a web address, your backend can check the signature without knowing any secret. Inside the token are claims such as `email`, `name`, `email_verified`, `aud` (which app it is for), `iss` (who issued it) and `exp` (when it expires).

### The flow

1. User clicks "Continue with Google" on `/login`
2. Google's pop-up asks the user to pick an account
3. Google gives the browser an ID token ("`credential`")
4. Angular -> `POST /api/auth/google { "credential": "<ID token>" }`
5. Backend checks the ID token:
  - signature (Google's public keys)
  - `iss` = `accounts.google.com`
  - `aud` = YOUR client id (so a token made for another app is refused)
  - `exp` = not expired
  - `email_verified` = true
6. Backend finds the user by email, or creates a CUSTOMER
7. Backend answers exactly like `/api/login`:  
{ `token` (your JWT), `role`, `userId`, `username`, `email` } + refresh cookie
8. Angular stores it like a normal login. Nothing else in the app changes.

**The Client ID.** Google gives your app a public "Client ID" (looks like `1234-abcd.apps.googleusercontent.com`). It is not a secret. It tells Google which app is asking, and your backend uses it in check `aud`. We do **not** use a Client Secret in this design, so there is no secret to leak.

---

## 2. Google Cloud Console, click by click (do this first, 20 minutes)

Google changes the names of these screens now and then. If yours look a little different, look for the words in **bold**.

1. Open <https://console.cloud.google.com> and sign in with a Gmail account (any one of the team can do this; use a normal Gmail, not a college or company account that may block it).
2. At the top, click the **project picker** (it shows "Select a project"). Click **New project**. Name: `elearning-platform`. Click **Create**. Then make sure this project is selected in the picker.
3. Open the left menu (three lines) -> **APIs & Services** -> **OAuth consent screen**. This opens the **Google Auth Platform** page. If it says "not configured yet", click **Get started**.
4. **App information**: App name `Online E-Learning Platform`; User support email: pick your Gmail. Click **Next**.

5. **Audience:** choose **External**. Click **Next**.
6. **Contact information:** type your email. Click **Next**.
7. Tick the box to agree to the user data policy, click **Continue**, then **Create**.
8. In the left menu of Google Auth Platform click **Clients**, then **+ Create client**.
9. **Application type:** choose **Web application**. **Name:** E-Learning Web.
10. Under **Authorized JavaScript origins** click **+ Add URI** and add these (one by one):
  - `http://localhost:8081`
  - `http://localhost`
  - `http://localhost:4200` (only if you sometimes run Angular on 4200)

Google's own guide says to add **both** `http://localhost` and `http://localhost:<port>` for local testing. 11. **Authorized redirect URIs:** leave it **empty**. This button does not use redirects. 12. Click **Create**. A box appears with **Client ID** and **Client secret**. - **Copy the Client ID** (it ends with `.apps.googleusercontent.com`). Keep it in a notepad. - You do **not** need the Client secret. Do not paste it anywhere. 13. In the left menu click **Audience**. The status is **Testing**. Under **Test users** click **+ Add users** and add the Gmail address of **every person who will log in with Google** (all 7 of you, your SME, the evaluator). Click **Save**. - While the app is in Testing, only these listed Gmail accounts can sign in (up to 100). Anyone else gets "Access blocked". - You can click **Publish app** later to let anyone sign in. For the basic email/profile sign-in this does not need Google's review, but Google may show an "unverified app" notice. 14. Wait about 5 minutes. New settings can take a few minutes to start working.

**When you deploy to a real website later:** come back to **Clients** and add that website address (for example `https://yoursite.com`) under Authorized JavaScript origins.

---

## 3. Backend changes

### 3a. `springapp/pom.xml` : add one dependency

Inside `<dependencies> ... </dependencies>`, right after the `spring-boot-starter-security` block, add:

```
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-oauth2-jose</artifactId>
</dependency>
```

No version number is needed: Spring Boot already knows the right version. After saving, reload Maven (in IntelliJ: the small "Reload" icon in the Maven panel; in Eclipse: right click project -> Maven -> Update Project). This library is what checks Google's token signature.

### 3b. `application.properties` : add at the very end

Do **not** replace the file. Only add these lines at the bottom:

```
# Login with Google: the public Client ID from Google Cloud Console (looks like 1234-
abcd.apps.googleusercontent.com).
# Paste yours after the colon, or set the GOOGLE_CLIENT_ID environment variable. Leave empty to switch
Google login off.
google.client-id=${GOOGLE_CLIENT_ID:}
```

Simplest for beginners: paste your Client ID after the colon, for example `google.client-id=${GOOGLE_CLIENT_ID:1234-abcd.apps.googleusercontent.com}`. A Client ID is a public identifier, not a password, so it is fine to keep it in the file. (Your database password is different: never commit that. Use the `DB_PASSWORD` environment variable.)

### 3x. `SecurityConfig.java`: nothing to do

The path `/api/auth/google` is already in the final `SecurityConfig.java` from the master plan (section 8). If you did not use that file, add `"/api/auth/google"` to the `permitAll` list of `/api/register`, `/api/login`, `...`.

### 3c. (Optional) `RateLimitFilter.java`: treat it like the other login calls

Your filter limits login attempts per minute. To include Google login, find this line (about line 38):

```
boolean authCall = path.equals("/api/login") || path.equals("/api/register") ||
path.equals("/api/refresh");
```

Change it to:

```
boolean authCall = path.equals("/api/login") || path.equals("/api/register") || path.equals("/api/refresh")
|| path.equals("/api/auth/google");
```

## 4. Complete code

`springapp/src/main/java/com/examly/springapp/model/GoogleLoginDTO.java` (new file)

```
package com.examly.springapp.model;

import jakarta.validation.constraints.NotBlank;

/**
 * Request body of POST /api/auth/google. "credential" is the Google ID token that the
 * "Sign in with Google" button gives the browser.
 */
public class GoogleLoginDTO {
    @NotBlank(message = "Google credential is required")
    private String credential;

    /** "login" (default) only lets an already registered person in; "register" creates the account first.
    */
    private String mode = "login";

    /** No-arg constructor required by Jackson. */
    public GoogleLoginDTO() {}

    public String getCredential() { return credential; }
    public void setCredential(String credential) { this.credential = credential; }
    public String getMode() { return mode; }
    public void setMode(String mode) { this.mode = mode; }
```

```
    public boolean isRegister() { return "register".equalsIgnoreCase(mode); }  
}
```

### springapp/src/main/java/com/examly/springapp/service/GoogleAuthService.java (new file)

```
package com.examly.springapp.service;  
  
import com.examly.springapp.exception.DatabaseOperationException;  
import com.examly.springapp.exception.DuplicateResourceException;  
import com.examly.springapp.exception.InvalidRequestException;  
import com.examly.springapp.exception.ResourceNotFoundException;  
import com.examly.springapp.model.User;  
import com.examly.springapp.repository.UserRepo;  
import java.util.Locale;  
import java.util.Set;  
import java.util.UUID;  
import org.slf4j.Logger;  
import org.slf4j.LoggerFactory;  
import org.springframework.beans.factory.annotation.Value;  
import org.springframework.dao.DataIntegrityViolationException;  
import org.springframework.security.access.AccessDeniedException;  
import org.springframework.security.authentication.BadCredentialsException;  
import org.springframework.security.crypto.password.PasswordEncoder;  
import org.springframework.security.oauth2.core.DelegatingOAuth2TokenValidator;  
import org.springframework.security.oauth2.core.OAuth2Error;  
import org.springframework.security.oauth2.core.OAuth2TokenValidator;  
import org.springframework.security.oauth2.core.OAuth2TokenValidatorResult;  
import org.springframework.security.oauth2.jwt.Jwt;  
import org.springframework.security.oauth2.jwt.JwtException;  
import org.springframework.security.oauth2.jwt.JwtValidators;  
import org.springframework.security.oauth2.jwt.NimbusJwtDecoder;  
import org.springframework.stereotype.Service;  
  
/**  
 * "Login with Google" (OAuth 2.0 / OpenID Connect).  
 * <p>  
 * The browser gives us a Google ID token. This class proves the token is real by checking  
 * Google's signature (public keys from Google), the issuer, the audience (our own client id)  
 * and the expiry. Then it finds the user with that e-mail, or creates a CUSTOMER account.  
 * The caller (GoogleAuthController) then issues the same JWT and refresh cookie as /api/login.  
 * <p>  
 * Nothing here is a secret: the client id is public. No password or Google token is logged.  
 */  
@Service  
public class GoogleAuthService {  
    private static final Logger log = LoggerFactory.getLogger(GoogleAuthService.class);  
    private static final String GOOGLE_KEYS_URL = "https://www.googleapis.com/oauth2/v3/certs";  
    private static final Set<String> GOOGLE_ISSUERS = Set.of("https://accounts.google.com",  
"accounts.google.com");  
    /** The users table requires a 10 digit mobile number; Google does not give one. The user can change it  
    later. */  
    private static final String PLACEHOLDER_MOBILE = "0000000000";  
  
    private final UserRepo userRepo;  
    private final PasswordEncoder passwordEncoder;  
    private final NimbusJwtDecoder decoder; // null when google.client-id is not set  
  
    public GoogleAuthService(UserRepo userRepo, PasswordEncoder passwordEncoder,  
        @Value("${google.client-id}") String clientId) {  
        this.userRepo = userRepo;  
        this.passwordEncoder = passwordEncoder;  
        this.decoder = clientId == null || clientId.isBlank() ? null : buildDecoder(clientId.trim());  
        if (this.decoder == null) {  
            log.warn("google.client-id is not set: Login with Google is switched off");  
        }  
    }  
  
    private static NimbusJwtDecoder buildDecoder(String clientId) {  
        NimbusJwtDecoder d = NimbusJwtDecoder.withJwkSetUri(GOOGLE_KEYS_URL).build();  
    }  
}
```

```

        OAuth2TokenValidator<Jwt> issuerCheck = jwt -> GOOGLE_ISSUERS.contains(jwt.getClaimAsString("iss"))
            ? OAuth2TokenValidatorResult.success()
            : OAuth2TokenValidatorResult.failure(new OAuth2Error("invalid_token", "Wrong token issuer",
null));
        OAuth2TokenValidator<Jwt> audienceCheck = jwt -> jwt.getAudience() != null &&
jwt.getAudience().contains(clientId)
            ? OAuth2TokenValidatorResult.success()
            : OAuth2TokenValidatorResult.failure(new OAuth2Error("invalid_token", "Token is for another
app", null));
        // createDefault() checks the expiry time; the other two checks are ours
        d.setJwtValidator(new DelegatingOAuth2TokenValidator<>(JwtValidators.createDefault(), issuerCheck,
audienceCheck));
        return d;
    }

    /**
     * "Log in with Google": the Google e-mail must already belong to a registered account. A person who
never registered gets
     * 404 "No account found ... register first" and nothing is stored.
     */
    public User loginWithGoogle(String credential) {
        log.trace("loginWithGoogle entered");
        Jwt jwt = verify(credential);
        String email = verifiedEmail(jwt);
        User user = DatabaseOperationException.guard("loading a user", () ->
userRepo.findByEmail(email)).orElse(null);
        if (user == null) {
            log.warn("Google login refused: no account for this Google e-mail");
            throw new ResourceNotFoundException("No account found for this Google e-mail. Please register
first.");
        }
        refuseAdmin(user);
        log.info("User {} logged in with Google", user.getUserId());
        return user;
    }

    /**
     * "Sign up with Google": creates a CUSTOMER account from the verified Google e-mail and name and
stores it in the users table.
     * If the e-mail is already registered the answer is 409 "already registered, log in instead" and
nothing changes.
     */
    public User registerWithGoogle(String credential) {
        log.trace("registerWithGoogle entered");
        Jwt jwt = verify(credential);
        String email = verifiedEmail(jwt);
        if (DatabaseOperationException.guard("loading a user", () ->
userRepo.findByEmail(email)).isPresent()) {
            log.warn("Google sign-up refused: the e-mail is already registered");
            throw new DuplicateResourceException("An account with this email already exists. Please log in
instead.");
        }
        return createCustomer(email, jwt.getClaimAsString("name"));
    }

    /** Checks the Google signature, issuer, audience and expiry. */
    private Jwt verify(String credential) {
        if (decoder == null) {
            throw new InvalidRequestException("Login with Google is not set up on this server");
        }
        try {
            return decoder.decode(credential);
        } catch (JwtException e) {
            log.warn("Google login refused: token could not be verified");
            throw new BadCredentialsException("Invalid Google token");
        }
    }

    private static String verifiedEmail(Jwt jwt) {
        String email = jwt.getClaimAsString("email");
        if (email == null || email.isBlank() ||

```

```

!Boolean.TRUE.equals(jwt.getClaimAsBoolean("email_verified"))) {
    log.warn("Google login refused: e-mail missing or not verified by Google");
    throw new BadCredentialsException("Google account e-mail is not verified");
}
return email.trim().toLowerCase(Locale.ROOT);
}

/** Business rule: admin accounts keep using the password login only (customers and instructors may use
Google). */
private static void refuseAdmin(User user) {
    if ("ADMIN".equals(user.getRole())) {
        log.warn("Google login refused for an admin account");
        throw new AccessDeniedException("Admin accounts must log in with their password");
    }
}

private User createCustomer(String email, String googleName) {
    String name = googleName == null || googleName.isBlank() ? email.substring(0, email.indexOf('@')) :
googleName.trim();
    if (name.length() > 50) {
        name = name.substring(0, 50);
    }
    // nobody knows this password, so the account can only be entered through Google (or "forgot
password" later)
    User user = new User(email, passwordEncoder.encode(UUID.randomUUID().toString()), name,
PLACEHOLDER_MOBILE, "CUSTOMER");
    try {
        User saved = userRepo.save(user);
        log.info("User {} created from Google login", saved.getUserId());
        return saved;
    } catch (DataIntegrityViolationException e) {
        // two Google logins for a new e-mail arrived at the same moment: use the one that won
        return userRepo.findByEmail(email)
            .orElseThrow(() -> new DuplicateResourceException("A user with this email already
exists"));
    }
}
}
}

```

## springapp/src/main/java/com/examly/springapp/controller/GoogleAuthController.java (new file)

```

package com.examly.springapp.controller;

import com.examly.springapp.config.JwtUtil;
import com.examly.springapp.model.GoogleLoginDTO;
import com.examly.springapp.model.User;
import com.examly.springapp.service.GoogleAuthService;
import com.examly.springapp.service.OtpService;
import com.examly.springapp.service.RefreshTokenService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import java.time.Duration;
import java.util.LinkedHashMap;
import java.util.Map;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseCookie;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

```

```

/**
 * POST /api/auth/google: exchanges a verified Google ID token for the same answer as /api/login
 * ({token, expiresIn, userId, role, username, email} plus the HttpOnly refresh cookie).
 */
@RestController
@RequestMapping("/api/auth")
@Tag(name = "Authentication", description = "Register, log in, refresh the access token and log out")
public class GoogleAuthController {
    private static final Logger log = LoggerFactory.getLogger(GoogleAuthController.class);
    private static final String REFRESH_COOKIE = "refresh_token";

    private final GoogleAuthService googleAuthService;
    private final JwtUtil jwtUtil;
    private final RefreshTokenService refreshTokenService;
    private final boolean secureCookie;
    private final long refreshDays;

    /** Day 1 mobile OTP (optional): a person who registered with the form and has not verified the phone
yet cannot skip it through Google. */
    @Autowired(required = false)
    private OtpService otpService;

    public GoogleAuthController(GoogleAuthService googleAuthService, JwtUtil jwtUtil,
                                RefreshTokenService refreshTokenService,
                                @Value("${app.cookie.secure:false}") boolean secureCookie,
                                @Value("${jwt.refresh-expiration-days:7}") long refreshDays) {
        this.googleAuthService = googleAuthService;
        this.jwtUtil = jwtUtil;
        this.refreshTokenService = refreshTokenService;
        this.secureCookie = secureCookie;
        this.refreshDays = refreshDays;
    }

    @Operation(summary = "Log in with Google",
                description = "Public. Body: {credential, mode}. credential = the Google ID token. mode \"login\" (default) works only for a registered e-mail (404 otherwise); mode \"register\" creates the CUSTOMER account (409 if it already exists).")
    @ApiResponse({
        @ApiResponse(responseCode = "200", description = "Logged in"),
        @ApiResponse(responseCode = "400", description = "Credential missing, or Google login is not configured"),
        @ApiResponse(responseCode = "401", description = "Google token is invalid, expired or the e-mail is not verified"),
        @ApiResponse(responseCode = "404", description = "mode login: no account is registered for this Google e-mail"),
        @ApiResponse(responseCode = "409", description = "mode register: this e-mail is already registered"),
        @ApiResponse(responseCode = "403", description = "Admin accounts cannot use Google login, or the mobile number is not verified yet")
    })
    @PostMapping("/google")
    public ResponseEntity<Map<String, Object>> google(@Valid @RequestBody GoogleLoginDTO body) {
        log.trace("google login called");
        User user = body.isRegister()
            ? googleAuthService.registerWithGoogle(body.getCredential()) // creates and stores the
account
            : googleAuthService.loginWithGoogle(body.getCredential()); // only for accounts that
already exist
        if (otpService != null) {
            otpService.requireVerified(user); // 403 "Please verify your mobile number to continue" while
the OTP is still pending
        }
        Map<String, Object> out = new LinkedHashMap<>();
        out.put("token", jwtUtil.generateToken(user.getEmail(), user.getRole(), user.getUserId(),
user.getUsername()));
        out.put("expiresIn", jwtUtil.getAccessExpirationMs() / 1000);
        out.put("userId", user.getUserId());
        out.put("role", user.getRole());
        out.put("username", user.getUsername());
        out.put("email", user.getEmail());
    }
}

```

```

String refresh = refreshTokenService.issue(user);
ResponseCookie cookie = ResponseCookie.from(REFRESH_COOKIE, refresh)
    .httpOnly(true)
    .secure(secureCookie)
    .sameSite("Strict")
    .path("/api")
    .maxAge(Duration.ofDays(refreshDays))
    .build();
return ResponseEntity.status(HttpStatus.OK).header(HttpHeaders.SET_COOKIE,
cookie.toString()).body(out);
}
}

```

## 5. Check it (backend only, no Google pop-up needed)

1. Start MySQL and the backend with `GOOGLE_CLIENT_ID` set to your Client ID. The console must **not** say `google.client-id is not set`.
2. In Swagger or curl: `POST /api/auth/google` with `{"credential":"abc"}`. The answer must be **401** (token invalid). If it is 404 the controller is not in the right package; if it is 403 the Security rule is missing.
3. Start again with `GOOGLE_CLIENT_ID` empty: the same call answers **400** "not set up". Good, that is the safe off switch.
4. The real test is in the frontend guide (needs the browser button).

## 6. If something goes wrong (backend)

| What you see                                                               | Why                                                                                        | Fix                                                      |
|----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|----------------------------------------------------------|
| <code>package org.springframework.security.oauth2...</code> does not exist | the pom dependency was not added or Maven did not reload                                   | add it (3a) and reload Maven                             |
| Backend log Google login refused: token could not be verified              | wrong Client ID, expired token, or the server cannot reach <code>www.googleapis.com</code> | same Client ID in backend and frontend; check internet   |
| 403 on <code>/api/auth/google</code> for a customer                        | the Security rule is missing, or the account is ADMIN (message says so)                    | use the final SecurityConfig                             |
| 403 "Please verify your mobile number"                                     | the person registered with the form and never verified (Day 1 rule)                        | verify on <code>/verify-phone</code> , then log in again |
| 400 "not set up"                                                           | <code>google.client-id</code> empty                                                        | set <code>GOOGLE_CLIENT_ID</code> and restart            |
| two rows for one person                                                    | the e-mail differs in case or spelling                                                     | the match is by e-mail; check the spelling               |

## 7. Security rules for the team

- Never commit the database password, JWT secret or any Google **Client secret**. This design needs **no** Client secret.
- The **Client ID** is public and may stay in code.

- The backend checks the token itself (signature, issuer, audience = your Client ID, expiry, verified e-mail). It never trusts an e-mail sent by the browser.
- Admin accounts cannot be reached through Google.
- Known limit: people are matched by e-mail (the `users` table has no Google id column). Fine for this project; a bigger system would also store Google's `sub`.